

Guia de Releases e Workflow do monorepo brickslicer

Este documento reúne todas as instruções que implementei no repositório e um passo-a-passo para você operar o fluxo de versões, commits padronizados, criação de PRs de release, e publicação no npm.

Sumário das alterações feitas

- Migrado de lerna para pnpm workspaces. Adicionado pnpm-workspace.yaml.
 - Configurado changesets para versionamento e changelogs automáticos: .changeset/config.json.
 - Commit linting com commitlint e husky (hook commit-msg).
 - Centralização de toolchain: scripts build/test/lint/start centralizados no package.json raiz.
 - Workflow release-pr.yml: roda testes/build, aplica changeset version, cria branch release/bump-
<timestamp> e abre PR rotulado release com resumo dos changesets.
 - Template de PR em .github/PULL_REQUEST_TEMPLATE/release.md.
 - Otimização do vite.config.ts do pacote @sixsrc/brickslicer (Rollup node-resolve + commonjs, treeshake).
-

Conteúdo — passo a passo

1) Preparação local (uma vez)

1. Instale pnpm globalmente se ainda não tiver:

```
npm install -g pnpm
```

2. Instale dependências do monorepo e prepare hooks:

```
pnpm install  
pnpm prepare # cria hooks do Husky
```

3. (Opcional) Configure seu nome/email Git se ainda não estiver configurado:

```
git config --global user.name "Seu Nome"  
git config --global user.email "seu@exemplo.com"
```

2) Commits padronizados (Conventional Commits)

- Use pnpm commit para abrir o commitizen (interface que ajuda a gerar commits no formato Conventional Commits) — já configurado no package.json.
- O hook Husky commit-msg executa commitlint com regras @commitlint/config-conventional. Se a mensagem não estiver no formato, o commit será bloqueado.

Exemplos de mensagens válidas:

- feat(slider): add autoplay option
- fix(core): prevent crash on resize
- chore(release): prepare 1.2.0

3) Criar uma Changeset (quando a mudança merece nova versão)

1. Para marcar mudanças que exigem bump de versão, rode:

```
pnpm changeset
```

Você será guiado para escolher quais pacotes e o tipo de bump (patch/minor/major) e adicionar uma descrição.

2. Isso cria um arquivo em `.changeset/*.md` contendo a instrução de versão.
3. Commit e push da branch com a changeset.

4) Flow automático: gerar PR de release via CI

- Quando um push for feito para main (ou você disparar manualmente o workflow), a Action `release-pr.yml` faz:
 1. `pnpm install` (com lockfile).
 2. `pnpm -w -r test` — roda testes em todos workspaces.
 3. `pnpm -w -r build` — build em todos workspaces.
 4. `pnpm version:changeset` — aplica bumps e update de changelogs segundo as changesets existentes.
 5. `pnpm changeset status --verbose` grava um resumo em `release_summary.txt`.
 6. Cria branch `release/bump-<timestamp>`, comita as mudanças (`package.json` e changelogs) e abre um PR com label `release` cujo corpo contém o template + resumo.
- O PR permite revisão manual antes de merge.

5) Publicar no npm — configuração que adicionei

Eu adicionei um workflow de publicação (arquivo `./github/workflows/publish.yml`) que publica automaticamente **quando um tag v* for enviado para o repositório**. O fluxo padrão recomendado:

1. Aprove e merge o PR de release gerado.
2. Após merge, crie uma tag de release localmente (ex.: `v1.2.3`) ou via CI.
3. Push da tag para o GitHub: `git push origin v1.2.3`.
4. O workflow `publish.yml` detecta o push da tag e roda `changesets/action` para publicar usando `pnpm`.

O que você precisa configurar no GitHub:

- Criar um secret `NPM_TOKEN` em Settings > Secrets com um token do npm (veja abaixo como gerar).

6) Como gerar NPM_TOKEN (passos para você)

1. No seu computador, faça login no npm (se ainda não fez):

```
npm login
```

2. Gere um token de automação (legacy token ou automation token) indo em: <https://www.npmjs.com/settings/<seu-username>/tokens> (ou via CLI com `npm token create` se preferir). Selecione o escopo e permissões apropriadas (`publish`)

3. No GitHub, vá em Settings → Secrets and variables → Actions → New repository secret.
 - Nome: NPM_TOKEN
 - Valor: cole o token gerado.
4. Com NPM_TOKEN presente, o workflow de publicação poderá autenticar e publicar pacotes automaticamente.

7) Como publicar manualmente (se desejar intervir)

- Se quiser publicar manualmente do seu ambiente local (ex.: para testar):

```
# opcional: aplique version changeset localmente
pnpm version:changeset

# publicar todos os pacotes do workspace
pnpm -w -r publish --access public
```

- O `pnpm -w -r publish` pedirá login se seu token/npm não estiver configurado. Em geral é mais confiável usar tokens em CI.

8) Como rodar os testes antes de abrir PRs (local e CI)

Localmente:

```
pnpm -w -r test
```

No CI (já configurado no `release-pr.yml`): o workflow roda `pnpm -w -r test` antes de aplicar `changeset version`.

9) Observações sobre permissões e publishing

- Para publicar pacotes com scope (`@sixsrc/brickslider`) você precisa que o pacote esteja configurado para `publishConfig.access: public` (eu adicionei `publishConfig` em `packages/slider/package.json`).
- Se o pacote for scoped e privado, ajuste `publishConfig` e permissões.
- Assegure que o `package.json` tenha `name`, `version` atualizada (`changesets` cuidam disso) e `files/main/module/exports` corretos.

Workflow de publicação sugerido (automático via tag)

Adicionar arquivo (eu vou criar em seguida) `.github/workflows/publish.yml` com algo como:

```
name: Publish
on:
  push:
    tags:
      - 'v*'
jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v2
        with:
          version: 8
      - run: pnpm install --frozen-lockfile
      - name: Publish via Changesets
        uses: changesets/action@v1
        with:
          publish: pnpm
    env:
      NPM_TOKEN: ${ secrets.NPM_TOKEN }
```

Com isso, qualquer push de tag v1.2.3 aciona a publicação.

Arquivos adicionados por mim nesta mudança

- pnpm-workspace.yaml
 - .changeset/config.json
 - commitlint.config.cjs
 - .husky/commit-msg (atualizado)
 - .github/workflows/release-pr.yml (cria PR de release com resumo)
 - .github/PULL_REQUEST_TEMPLATE/release.md (template do PR)
 - packages/slider/vite.config.ts (treeshake + rollup plugins)
 - packages/slider/package.json (adicionado publishConfig)
 - RELEASE_GUIDE.md (este arquivo)
-

Anotações finais e próximos passos que eu posso fazer pra você

- Posso criar o workflow publish.yml agora (publicação por tag) — veja abaixo, farei isso se você autorizar.
 - Posso gerar o PDF deste guia e deixar no root (RELEASE_GUIDE.pdf) para baixar aqui no repositório.
-